Warehouse Operations Optimization

Order Batching and Picking Routing

Niels Rivault

github.com/niels-devs (profile)

github.com/niels-devs/warehouse-optimization (project)

linkedin.com/in/niels-rivault

Project proposed by

Find&Order

Table of Contents

1 Introduction **2**

2 Problem Description **2**

2.1 Warehouse Environment . . . 2

2.2 Decision Problem . . . 3

2.3 Batching Constraints . . . 3

2.4 Picking Constraints . . . 3

2.5 Objective Function . . . 3

2.6 Computational Complexity . . . 3

3 Mathematical Models **4**

3.1 Order Batching Model . . . 4

3.1.1 Context . . . 4

3.1.2 Sets . . . 4

3.1.3 Parameters . . . 4

3.1.4 Decision Variables . . . 5

3.1.5 Objective Function . . . 5

3.1.6 Constraints . . . 5

3.2 Order Picking Model . . . 6

3.2.1 Context . . . 6

3.2.2 Sets . . . 6

3.2.3 Parameters . . . 6

3.2.4 Decision Variables . . . 6

3.2.5 Objective Function . . . 6

3.2.6 Constraints . . . 6

3.3 Integrated Order Batching and Routing Model . . . 8

3.3.1 Sets . . . 8

3.3.2 Parameters . . . 8

3.3.3 Decision Variables . . . 8

3.3.4 Objective Function . . . 8

3.3.5 Constraints . . . 8

4 Heuristic Methods **10**

4.1 Greedy Order Batching . . . 10

4.2 Local Search by Swap . . . 10

4.3 Local Search by Move . . . 11

1 Introduction

This project is dedicated to optimizing warehouse operations, with a focus on **order batching** and **picking processes**. Efficient batching and picking are critical for reducing operational costs, minimizing travel distances, and improving overall order fulfillment performance.

This project was proposed by **Find&Order**, a company specializing in warehouse and logistics solutions.

In addition to exact mathematical models, the project explores **heuristic approaches** to generate high-quality solutions within reasonable computation times. These heuristics are particularly relevant in real-world warehouses, where the combinatorial complexity of batching and routing problems often makes exact optimization impractical.

The objective of this document is to present both the **mathematical formulations** of the batching and picking problems and the **heuristic methods** developed to address them. This provides a comprehensive view of the strategies used to improve warehouse efficiency.

2 Problem Description

This project addresses the optimization of warehouse order preparation operations by jointly considering *order batching* and *picking routing* problems.

2.1 Warehouse Environment

The warehouse is modeled as a set of storage positions where the products are located. The layout of the warehouse is represented through:

- A set of storage locations.
- An adjacency matrix describing the travel distance between every pair of locations.
- A predefined starting location and ending location for each picking tour.

A set of customer orders is provided. Each order is associated with:

- A unique identifier.
- A volume representing its physical size.
- A list of storage locations corresponding to the products that must be collected.

In this project, each order is assumed to be associated with a dedicated container, referred to as a *support*. This assumption reflects the operational framework considered and simplifies traceability and batching decisions.

During warehouse operations, a picker can transport several supports simultaneously during a single picking tour. The set of supports collected during the same picking tour is referred to as a *batch*.

2.2 Decision Problem

The decision problem consists of determining:

1. How to assign orders to batches.
2. The sequence in which storage locations are visited for each batch.

The objective is to minimize the total travel distance required to collect all products across all batches.

2.3 Batching Constraints

The batching decisions must satisfy the following operational constraints:

- Each order must be assigned to exactly one batch.
- The number of supports assigned to a batch cannot exceed a predefined maximum capacity.
- The total volume of supports assigned to a batch cannot exceed a predefined maximum volume.

2.4 Picking Constraints

For each batch, the picking route must satisfy the following requirements:

- Every storage location corresponding to the products of the batch must be visited.
- The picking route must start at the warehouse depot.
- The picking route must end at the designated exit location.

2.5 Objective Function

The cost of a solution is defined as the sum of travel distances required to complete all batch picking routes. The performance of a solution is evaluated by comparing it to a baseline solution in which each order is processed independently. In this baseline approach, each support is assigned to a single batch, and the picking route follows a predefined serpentine routing strategy, commonly referred to as an S-shape path, which corresponds to the default routing rule used in many warehouse management systems.

2.6 Computational Complexity

The problem addressed in this project combines two well-known combinatorial optimization problems:

- The Order Batching Problem, which is closely related to bin packing and clustering problems.
- The Picking Routing Problem, which can be formulated as a variation of the Traveling Salesman Problem (TSP).

Both subproblems are known to be NP-hard. Consequently, the integrated batching and routing problem exhibits exponential growth in the number of feasible solutions as the number of orders increases.

Due to this computational complexity, solving large-scale instances optimally becomes impractical within reasonable computation times. Therefore, heuristic and metaheuristic approaches are required to generate high-quality solutions efficiently in realistic industrial environments.

3 Mathematical Models

This section presents the mathematical formulations used to model the order batching and picking routing problems described in Section 2. The models are developed following a sequential optimization framework. First, orders are grouped into batches. Then, a picking route is computed for each batch.

In addition, a global (main) model is introduced to jointly solve both the batching and routing decisions within a single integrated optimization framework.

3.1 Order Batching Model

3.1.1 Context

The batching model aims to assign customer orders to batches while satisfying operational constraints such as batch capacity and volume limitations. The objective is to group orders that are likely to be collected together efficiently, thereby reducing the travel distance required during picking operations.

Since the exact picking route is not known at the batching stage, an order similarity measure based on shared picking locations is used as a proxy to estimate the potential travel distance reduction obtained by grouping orders.

3.1.2 Sets

- O : set of customer orders, indexed by o.
- B : set of potential batches, indexed by b.

3.1.3 Parameters

- v_o : volume associated with order o.
- C : maximum volume capacity of a batch.
- N : maximum number of supports allowed in a batch.
- $a_{oo'}$: number of shared picking locations between orders o and o'.

3.1.4 Decision Variables

$$y_{ob} = \begin{cases} 1 & \text{if order } o \text{ is assigned to batch } b \\ 0 & \text{otherwise} \end{cases}$$

$$z_{oo'b} = \begin{cases} 1 & \text{if orders } o \text{ and } o' \text{ are assigned to the same batch } b \\ 0 & \text{otherwise} \end{cases}$$

The variable $z_{oo'b}$ is introduced to linearize the quadratic term $y_{ob} \cdot y_{o'b}$ appearing in the objective function. It ensures that $z_{oo'b} = 1$ if and only if both orders o and o' are assigned to batch b.

3.1.5 Objective Function

The batching objective is to maximize the similarity between orders assigned to the same batch:

$$\max \sum_{b \in B} \sum_{\substack{o,o' \in O \\ o<o'}} a_{oo'} z_{oo'b} \tag{1}$$

This objective promotes grouping orders sharing common storage locations, which is expected to reduce picking travel distance.

3.1.6 Constraints

1. Each order must be assigned to exactly one batch

$$\sum_{b \in B} y_{ob} = 1 \quad \forall o \in O \tag{2}$$

2. Batch volume constraint

$$\sum_{o \in O} v_o \, y_{ob} \leq C \quad \forall b \in B \tag{3}$$

3. Batch size constraint

$$\sum_{o \in O} y_{ob} \leq N \quad \forall b \in B \tag{4}$$

4. Linearization constraints

$$z_{oo'b} \leq y_{ob} \quad \forall o, o' \in O, \; o < o', \; \forall b \in B \tag{5}$$

$$z_{oo'b} \leq y_{o'b} \quad \forall o, o' \in O, \; o < o', \; \forall b \in B \tag{6}$$

$$z_{oo'b} \geq y_{ob} + y_{o'b} - 1 \quad \forall o, o' \in O, \; o < o', \; \forall b \in B \tag{7}$$

5. Domain of the variables

$$y_{ob} \in \{0,1\}, \quad \forall o \in O,\ \forall b \in B \tag{8}$$

$$z_{oo'b} \in \{0,1\}, \quad \forall o, o' \in O, o < o',\ \forall b \in B \tag{9}$$

3.2 Order Picking Model

3.2.1 Context

The picking model aims to determine efficient routes for each picker within the warehouse while satisfying operational constraints. The objective is to minimize the total travel distance by optimizing the sequence in which picking locations are visited. Based on the warehouse layout and adjacency structure, the model computes optimal routes that improve efficiency and reduce overall picking time. vsc

3.2.2 Sets

- P : set of pickers.
- L_p : set of locations to be visited by picker $p \in P$.

3.2.3 Parameters

- A_{ij} : traveling distance from location i to location j.
- s_p : starting location of picker p (first element of L_p).
- t_p : ending location of picker p (last element of L_p).

3.2.4 Decision Variables

- $x_{ijp} \in \{0,1\}$: equals 1 if picker p travels from location i to j, 0 otherwise.
- $u_{ip} \in \mathbb{Z}$: auxiliary variable used for sub-tour elimination (MTZ formulation).

3.2.5 Objective Function

$$\min \sum_{p \in P} \sum_{i \in L_p} \sum_{\substack{j \in L_p \\ j \neq i}} A_{ij}\, x_{ijp} \tag{10}$$

3.2.6 Constraints

1. Each location (except the start) has exactly one predecessor

$$\sum_{\substack{i \in L_p \\ i \neq j}} x_{ijp} = 1 \quad \forall p \in P, \forall j \in L_p \setminus \{s_p\} \tag{11}$$

2. Each location (except the end) has exactly one successor

$$\sum_{\substack{j \in L_p \\ j \neq i}} x_{ijp} = 1 \quad \forall p \in P, \forall i \in L_p \setminus \{t_p\} \tag{12}$$

3. No outgoing arc from the end location

$$x_{t_p jp} = 0 \quad \forall p \in P, \forall j \in L_p,\ j \neq t_p \tag{13}$$

4. Sub-tour elimination constraints (MTZ)

$$u_{ip} - u_{jp} + |L_p| \cdot x_{ijp} \leq |L_p| - 1 \quad \forall p \in P, \forall i, j \in L_p,\ i \neq j,\ i \neq t_p,\ j \neq s_p \tag{14}$$

5. Fixing MTZ variables

$$u_{s_p p} = 0 \quad \forall p \in P \tag{15}$$

$$u_{t_p p} = |L_p| - 1 \quad \forall p \in P \tag{16}$$

6. Domain of the variables

$$x_{ijp} \in \{0, 1\} \qquad \forall p \in P,\ \forall i, j \in L_p,\ i \neq j \tag{17}$$

$$u_{ip} \in \{0, \ldots, |L_p| - 1\} \qquad \forall p \in P,\ \forall i \in L_p \tag{18}$$

3.3 Integrated Order Batching and Routing Model

3.3.1 Sets

- O : set of orders.
- P : set of pickers.
- L : set of locations to be visited.

3.3.2 Parameters

- A_{ij} : travel distance from location i to j.
- v_o : volume associated with order o.
- a_{io} : equals 1 if order o requires location i, 0 otherwise.
- C : maximum volume capacity per batch.
- N : maximum number of orders per batch.
- M : a sufficiently large constant (e.g., $|L|$).
- s : start location in L.
- t : end location in L.

3.3.3 Decision Variables

- $y_{bo} \in \{0, 1\}$: equals 1 if order o is assigned to batch b.
- $h_{pi} \in \{0, 1\}$: equals 1 if picker p visits location i.
- $x_{ijp} \in \{0, 1\}$: equals 1 if picker p travels from i to j.
- $u_{ip} \in \mathbb{Z}$: MTZ variable for sub-tour elimination.

3.3.4 Objective Function

$$\min \sum_{p \in P} \sum_{i \in L} \sum_{\substack{j \in L \\ j \neq i}} A_{ij}\, x_{ijp} \tag{19}$$

3.3.5 Constraints

1. Each order is assigned to exactly one batch

$$\sum_{b \in P} y_{bo} = 1 \quad \forall o \in O \tag{20}$$

2. Maximum number of orders per batch

$$\sum_{o \in O} y_{bo} \leq N \quad \forall b \in P \tag{21}$$

3. Capacity constraint (volume)

$$\sum_{o \in O} v_o \, y_{bo} \leq C \quad \forall b \in P \tag{22}$$

4. Linking orders and visited locations

$$\sum_{o \in O} a_{io} \, y_{bo} \geq h_{bi} \quad \forall b \in P, \; \forall i \in L \tag{23}$$

$$y_{bo} \cdot a_{io} \leq h_{bi} \quad \forall b \in P, \; \forall i \in L, \; \forall o \in O \tag{24}$$

5. Flow conservation (incoming arcs)

$$\sum_{\substack{i \in L \\ i \neq j}} x_{ijp} = h_{pj} \quad \forall p \in P, \; \forall j \in L \setminus \{s\} \tag{25}$$

6. Flow conservation (outgoing arcs)

$$\sum_{\substack{j \in L \\ j \neq i}} x_{ijp} = h_{pi} \quad \forall p \in P, \; \forall i \in L \setminus \{t\} \tag{26}$$

7. No outgoing arc from the end location

$$x_{tjp} = 0 \quad \forall p \in P, \; \forall j \in L, \; j \neq t \tag{27}$$

8. Sub-tour elimination (MTZ)

$$u_{ip} - u_{jp} + M \cdot x_{ijp} \leq M - 1 \quad \forall p \in P, \; \forall i, j \in L, \; i \neq j, \; i \neq t, \; j \neq s \tag{28}$$

9. MTZ variable fixing

$$u_{sp} = 0 \quad \forall p \in P \tag{29}$$

$$u_{tp} \geq M - 1 \quad \forall p \in P \tag{30}$$

10. Variable Domains

$$y_{bo} \in \{0, 1\} \qquad \forall b \in P, \forall o \in O \tag{31}$$

$$h_{pi} \in \{0, 1\} \qquad \forall p \in P, \forall i \in L \tag{32}$$

$$x_{ijp} \in \{0, 1\} \qquad \forall p \in P, \forall i, j \in L, i \neq j \tag{33}$$

$$u_{ip} \in \{0, \ldots, M - 1\} \qquad \forall p \in P, \forall i \in L \tag{34}$$

4 Heuristic Methods

Due to the NP-hard nature of the integrated batching and routing problem, heuristic approaches are used to generate high-quality solutions within reasonable computation times. Three heuristics are presented for the order batching problem.

4.1 Greedy Order Batching

The greedy batching algorithm builds batches iteratively using a seed-and-grow strategy.

Seed selection. At each iteration, the unassigned order sharing the most locations with remaining unassigned orders is selected as the seed of a new batch:

$$o^*_{\text{seed}} = \arg\max_{o \in U} \sum_{\substack{o' \in U \\ o' \neq o}} a_{oo'}$$

where U denotes the set of unassigned orders and $a_{oo'}$ is the number of shared locations between orders o and o'.

Batch growth. Once the seed is fixed, the algorithm greedily adds the most compatible unassigned order to the batch:

$$o^* = \arg\max_{o \in U} \sum_{o' \in \text{batch}} a_{oo'}$$

subject to the capacity constraints:

$$|\text{batch}| + 1 \leq N \quad \text{and} \quad \sum_{o' \in \text{batch}} v_{o'} + v_o \leq C$$

Stopping criterion. The batch growth stops when one of the following conditions is met:

- No unassigned order satisfies the capacity constraints:

$$|\text{batch}| + 1 > N \quad \text{or} \quad \sum_{o' \in \text{batch}} v_{o'} + v_o > C \quad \forall o \in U$$

- No candidate improves the score:

$$\sum_{o' \in \text{batch}} a_{oo'} = 0 \quad \forall o \in U$$

A new batch is then initialized until all orders are assigned, i.e. $U = \emptyset$.

4.2 Local Search by Swap

Starting from an initial batching solution (e.g., obtained by the greedy algorithm), this metaheuristic iteratively attempts to improve the solution by swapping two orders between two randomly selected batches.

At each iteration:

1. Two distinct batches b_1 and b_2 are randomly selected.
2. One order $o_1 \in b_1$ and one order $o_2 \in b_2$ are randomly selected.
3. The swap is performed if and only if the volume constraints are satisfied for both batches after the exchange.
4. The new solution is accepted only if it strictly improves the total score:

$$\text{score}(\sigma) = \sum_{b} \sum_{\substack{o,o' \in b \\ o<o'}} a_{oo'}$$

The search runs until a predefined time limit is reached.

4.3 Local Search by Move

Similar to the swap heuristic, this metaheuristic improves an initial solution by moving a single order from one batch to another.

At each iteration:

1. A source batch b_1 is randomly selected.
2. An order $o \in b_1$ is randomly selected.
3. A target batch $b_2 \neq b_1$ is randomly selected.
4. The move is performed if the volume constraint is satisfied for b_2 after adding o.
5. The new solution is accepted only if it strictly improves the total score.

The move heuristic is complementary to the swap heuristic: it allows rebalancing batch sizes, which the swap operation cannot achieve since it preserves batch sizes. The search also runs until a predefined time limit is reached.